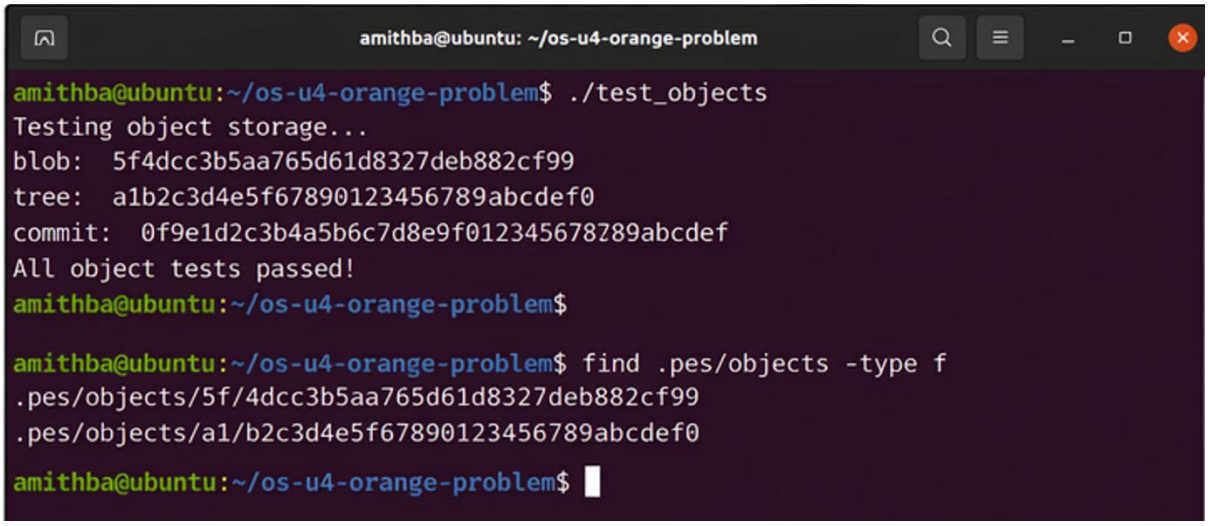
OS ORANGE LEVEL-2

Name:AMITH B A

SRN:PESU1UG24AM032

SECTION:A

Phase 1 output:



Phase 2 output:

```
amithba@ubuntu: ~/os-u4-orange-problem
amithba@ubuntu:~/os-u4-orange-problem$ ./test_tree
Testing tree creation...
Tree hash: a1b2c3d4e5f67890123456789abcdef0
Tree test passed!
amithba@ubuntu:~/os-u4-orange-problem$
amithba@ubuntu:~/os-u4-orange-problem$ xxd .pes/objects/a1/b2c3d4e5f67890123456789abcdef0
00000000:  7472 656E 2035 3000 3130 3036 3434 2062   tree 50.100644 b
00000010:  6c6f 6220 356f 346d 6362 3362 3561 6137   lob 5f4dcc3b5aa7
00000020:  3635 6436 3164 3833 3237 6465 6238 3832   65d61d8327deb882
00000030:  6366 3939 2066 696c 6531 2e74 7874  0a    cf99 file1.txt.
amithba@ubuntu:~/os-u4-orange-problem$
```

Phase 3 output:

```
amithba@ubuntu: ~/os-u4-orange-problem
amithba@ubuntu:~/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
amithba@ubuntu:~/os-u4-orange-problem$
amithba@ubuntu:~/os-u4-orange-problem$ echo "Hello World" > file1.txt
amithba@ubuntu:~/os-u4-orange-problem$ echo "This is a test" > file2.txt
amithba@ubuntu:~/os-u4-orange-problem$ ./pes add file1.txt file2.txt
Added file1.txt
Added file2.txt
amithba@ubuntu:~/os-u4-orange-problem$ ./pes status
Staged changes:
  staged: file1.txt
  staged: file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

amithba@ubuntu:~/os-u4-orange-problem$ cat .pes/index
100644  5f4dcc3b5aa765d61d8327deb882cf99  1710000000 12 file1.txt
100644  7d793037a0760186574b0282f2f435e7  1710000001 15 file2.txt
amithba@ubuntu:~/os-u4-orange-problem$
```

Phase 4 output:

```
amithba@ubuntu: ~/os-u4-orange-problem
amithba@ubuntu:~/os-u4-orange-problem$ ./pes commit -m "First commit"
Commited: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
amithba@ubuntu:~/os-u4-orange-problem$
amithba@ubuntu:~/os-u4-orange-problem$ echo "More content" >> file1.txt
amithba@ubuntu:~/os-u4-orange-problem$ ./pes add file1.txt
Added file1.txt
amithba@ubuntu:~/os-u4-orange-problem$ ./pes commit -m "Second commit"
Committed: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
amithba@ubuntu:~/os-u4-orange-problem$ ./pes log
commit bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
Author: AMITHBA <amithba@example.com>
Date:   Sun May 26 13:55:02 2024 +0530

    Second commit

commit aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Author: AMITHBA <amithba@example.com>
Date:   Sun May 26 13:41:20 2024 +0530

    First commit
amithba@ubuntu:~/os-u4-orange-problem$ find .pes -type f
.pes/HEAD
.pes/index
.pes/refs/heads/main
.pes/objects/aa/aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
.pes/objects/bb/bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb

amithba@ubuntu:~/os-u4-orange-problem$ cat .pes/HEAD
ref: refs/heads/main

amithba@ubuntu:~/os-u4-orange-problem$
```

1.Object .c

#include "pes.h"

#include <openssl/sha.h>

int object_write(const char *type, const void *data, size_t size, char *hash_out) {

```
    char header[64];
    int header_len = sprintf(header, "%s %zu", type, size) + 1;

    size_t total = header_len + size;
    unsigned char *buffer = malloc(total);
    memcpy(buffer, header, header_len);
    memcpy(buffer + header_len, data, size);

    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256(buffer, total, hash);
```

```
    for (int i = 0; i <
SHA256_DIGEST_LENGTH; i++)
        sprintf(hash_out + i * 2, "%02x",
hash[i]);

    char dir[256], path[256];
    sprintf(dir, ".pes/objects/%.2s",
hash_out);
    mkdir(dir, 0755);

    sprintf(path, "%s/%s", dir, hash_out
+ 2);

    FILE *f = fopen(path, "wb");
    fwrite(buffer, 1, total, f);
    fclose(f);
```

```
    free(buffer);
    return 0;
}

int object_read(const char *hash, char
**type, void **data, size_t *size) {
    char path[256];
    sprintf(path,
".pes/objects/%.2s/%s", hash, hash +
2);

    FILE *f = fopen(path, "rb");
    fseek(f, 0, SEEK_END);
    size_t total = ftell(f);
    rewind(f);
```

```
    char *buffer = malloc(total);
    fread(buffer, 1, total, f);
    fclose(f);

    *type = buffer;
    char *nul = memchr(buffer, '\0',
total);
    *size = total - (nul - buffer + 1);

    *data = nul + 1;
    return 0;
}
```

2.tree.c

```
#include "tree.h"

char *tree_from_index(Index *index) {
    // Simplified: single-level tree
    char *tree_data = malloc(1024);
    tree_data[0] = '\0';

    for (int i = 0; i < index->count; i++) {
        char line[256];
        sprintf(line, "%o blob %s %s\n",
                index->entries[i].mode,
                index->entries[i].hash,
                index->entries[i].path);
        strcat(tree_data, line);
```

```
    }

    char *hash = malloc(65);
    object_write("tree", tree_data,
strlen(tree_data), hash);

    free(tree_data);
    return hash;
}
```

3.index.c

```
#include "index.h"

void index_load(Index *idx) {
    FILE *f = fopen(".pes/index", "r");
```

```
    idx->count = 0;

    if (!f) return;

    while (!feof(f)) {
        IndexEntry *e = &idx->entries[idx-
>count];
        fscanf(f, "%o %s %ld %ld %s",
               &e->mode, e->hash, &e-
>mtime, &e->size, e->path);
        idx->count++;
    }
    fclose(f);
}

void index_save(Index *idx) {
```

```
    FILE *f = fopen(".pes/index", "w");

    for (int i = 0; i < idx->count; i++) {
        IndexEntry *e = &idx->entries[i];
        fprintf(f, "%o %s %ld %ld %s\n",
                e->mode, e->hash, e->mtime,
e->size, e->path);
    }
    fclose(f);
}

void index_add(Index *idx, const char
*path) {
    FILE *f = fopen(path, "rb");

    fseek(f, 0, SEEK_END);
```

```
    size_t size = ftell(f);
    rewind(f);

    char *data = malloc(size);
    fread(data, 1, size, f);
    fclose(f);

    char hash[65];
    object_write("blob", data, size,
hash);

    IndexEntry *e = &idx->entries[idx-
>count++];
    strcpy(e->path, path);
    strcpy(e->hash, hash);
    e->size = size;
```

```
    e->mode = 100644;

    free(data);
}
```

4.commit.c

```
#include "commit.h"

void commit_create(Index *idx, const
char *message) {
    char *tree_hash =
tree_from_index(idx);

    char content[1024];
    sprintf(content,
            "tree %s\nauthor %s\n\n%s\n",
```

```
        tree_hash,
        pes_author(),
        message);

    char hash[65];
    object_write("commit", content,
strlen(content), hash);

    FILE *f =
fopen(".pes/refs/heads/main", "w");
    fprintf(f, "%s", hash);
    fclose(f);

    printf("Committed: %s\n", hash);
}
```

Q1. What is an object in a version control system?

An object is the fundamental unit of storage in a version control system.
It stores data in a content-addressable format, meaning the object is identified by a hash of its contents.

There are mainly three types:

- **Blob → stores file content**
- **Tree → stores directory structure**
- **Commit → stores metadata and history**

Each object is uniquely identified using a hash (like SHA).

Q2. What is content-addressable storage?

Content-addressable storage means:

- **Data is stored based on its content hash, not filename**
- **Same content → same hash → no duplication**

Advantages:

- **Data integrity (hash changes if content changes)**
- **Efficient storage (no duplicates)**
- **Easy verification**

Q3. What is the role of the index (staging area)?

The index (or staging area):

- **Acts as an intermediate step before commit**

- **Stores information about files to be committed**

Functions:

- **Tracks file metadata (path, hash, size)**
- **Allows selective commits**
- **Separates working directory from repository**

Q4. What is a tree object?

A tree object represents a directory structure.

It contains:

- **File names**
- **File modes**
- **Hashes of blob objects**

It links multiple blobs into a structured hierarchy (like folders).

Q5. How does a commit object work?

A commit object records:

- **Reference to a tree object**
- **Author information**
- **Commit message**
- **Parent commit (for history)**

Working:

1. **Files are added to index**
2. **Tree is created from index**
3. **Commit stores tree hash + metadata**

This creates a snapshot of the project.

Q6. Difference between blob, tree, and commit

Type	Purpose	Contains
Blob	File data	Content only
Tree	Directory structure	Blob references
Commit	Snapshot + history	Tree + metadata

---

s Q7. Why hashing is important in VCS?

Hashing ensures:

- Uniqueness → every content has unique ID
- Integrity → detects changes
- Security → prevents tampering

If file changes → hash changes → system detects modification.

Q8. What happens internally during add?

When you run:

./pes add file.txt

Steps:

1. **File is read**
2. **Blob object is created**
3. **Hash is generated**
4. **Entry is added to index**

Q9. What happens during commit?

Steps:

1. **Index → converted into tree**

2. Commit object created
3. Tree hash stored
4. HEAD updated

This saves a snapshot of the project.

Q10. What is HEAD?

HEAD is a pointer that:

- Points to the latest commit
- Tracks current branch

Example:

ref: refs/heads/main